



# Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



## Angular Signals

### What are signals?

"Angular Signals is a system that granularly tracks how and where your state is used throughout an application, allowing the framework to optimize rendering updates." – **Angular Official Website**

A signal is a wrapper around a value that notifies interested consumers when that value changes. Signals can contain any value, from primitives to complex data structures. You read a signal's value by calling its getter function, which allows Angular to track where the signal is used.

Traditionally, Angular applications have relied heavily on:

- Component properties
- Event binding
- RxJS Observables
- async pipe
- Angular change detection

For example:

```
export class TraditionalCounter {  
  count = 0;  
  
  increment() {  
    this.count++;  
  }  
  decrement() {  
    this.count--;  
  }  
  reset() {  
    this.count = 0;  
  }  
}
```

This works perfectly well. But as applications become larger, Angular needs a precise way of answering:

"Which piece of UI depends on which piece of state?"

Signals provide a reactive state primitive that allows Angular to track dependencies between state and consumers of that state. Signal improves Angular's change detection mechanism, which also improves performance, and makes our code more reactive. So, a Signal is a **Reactive Container** that holds a value. Think of signal as it:

- Stores a value
- Knows who is using the value
- Notifies Angular when the value changes





# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**



## Understand Problem First

Let's start with a simple example without the use of signals. Say you are writing code to perform some basic math operations. Consider code below:

```
export class Samplecomponent {  
  x = 5;  
  y = 3;  
  z = this.x + this.y;  
  constructor() {  
    console.log(this.z);  
  }  
}
```

What does this code log to the console? Yes, you are right, it logs out 8.

Sometime later in the code, we change the value of x. consider code now:

```
export class Samplecomponent {  
  x = 5;  
  y = 3;  
  z = this.x + this.y;  
  constructor() {  
    console.log(this.z);  
    this.change();  
  }  
  change() {  
    this.x = 10;  
    console.log(this.z);  
  }  
}
```

What does z log out now? It still logs out 8! That's because a value is assigned to z when the expression is first evaluated. The z variable does not react to changes in x or y. But we want our variables to react to changes!

One of the reasons we use Angular is to build reactive websites: For an example, when the user updates the quantity, the related variables (such as subtotal and tax) should react and adjust the costs. If the user selects to delete an item from the cart, we again want the related variables to react and correctly recalculate the costs.

| Cart               |            |          |        |
|--------------------|------------|----------|--------|
| Vehicle            | Price      | Quantity |        |
| AT-AT              | 71,099.68  | 2        | Delete |
| TIE/LN starfighter | 19,416.13  | 1        | Delete |
| Subtotal:          | 161,615.49 |          |        |
| Delivery:          | Free       |          |        |
| Estimated Tax:     | 17,373.66  |          |        |
| Total:             | 178,989.15 |          |        |





# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**



With signals, our code can be more reactive. Our prior example implemented with signals would look like this:

```
import { Component, computed, signal } from '@angular/core';

@Component({
  selector: 'app-signalcomp',
  imports: [],
  templateUrl: './signalcomp.html',
  styleUrls: ['./signalcomp.css'],
})
export class Signalcomp {
  x = signal(5);
  y = signal(3);
  z = computed(() => this.x() + this.y());
  constructor() {
    console.log(this.z()); // 8
    this.change();
  }
  change() {
    this.x.set(10);
    console.log(this.z()); // 13
  }
}
```

The code above defines two signals: x and y and gives them initial values of 5 and 3. We then define a computed signal, z, which is the sum of x and y. Since signals provide change notifications, when the x or y signals change, any values computed from those signals will automatically recalculate. This code is now reactive!

Computed signals react and recalculate when any of its dependent signals change. If a signal is bound in a template, when the signal changes, Angular's change detection automatically updates any view that reads the signal. And the user sees the changed value.

In nut-shell, you can think of a signal as a value plus a change notification. A signal is just a special type of variable that holds a value. But unlike other variables, a signal also provides notification when the variable value changes.

## Why Do We Need Signals?

While learning signals, first question might raise in mind:

" Why do we need Signals when Angular already has data binding?".





# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**



Angular Signals exist to revolutionize how Angular detects changes, not to replace the data binding syntax itself. While you still use data binding (or) to display information in your HTML templates, **Signals completely change how the framework finds out that a value has changed.**

Before Signals, Angular had to guess when data changed; with Signals, Angular is told exactly what changed and where.

## 1. Fine-Grained Reactivity vs. Zone.js Guesswork

- **The Old Way (Zone.js):** Traditional Angular data binding relies on a library called. Whenever an asynchronous event occurs (like a click, timer, or HTTP request), Zone.js intercepts it and tells Angular, *"Something might have changed somewhere."* Angular then has to scan your entire component tree from top to bottom to check for updates. This is known as **coarse-grained** reactivity and can cause performance bottlenecks in large applications.
- **The New Way (Signals):** Angular Signals create a direct producer-consumer relationship between your data and the template. When a Signal value updates, it tells Angular exactly which specific component and HTML node needs to re-render. The framework bypasses unnecessary dirty-checking for the rest of the application.

## 2. Glitch-Free Computed State

In traditional data binding, creating a variable that depends on other variables (e.g., ) requires manual management or heavy method calls in the template. If you use a method in a traditional template, it executes on every single change detection cycle, destroying performance.

- Signals introduce.
- Computed signals track their own dependencies automatically.
- They are lazily evaluated and cached. They only recalculate when their underlying dependencies actually change.

## 3. Cleaner Component Communication

Data binding between parent and child components historically required verbose decorators and paired properties to achieve two-way binding. Signals vastly simplify this syntax:

- **Signal Inputs:** Declaring properties treats inputs as read-only signals that can seamlessly feed into logic.
- **Model Inputs:** The function automatically creates a two-way signal binding, cutting down boilerplate code into a single line.

For many years, Angular developers tried to avoid NgRx by using RxJS Subjects and complex observable pipelines, but that also gets difficult. You need to understand many operators, how they work and even learn marble diagrams. For most applications, especially small- to medium-sized ones, you don't need all this complexity.

With the introduction of Signals and modern data-fetching APIs like resource and httpResource, Angular now offers a simpler, more intuitive state management approach built directly into the framework.





# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**



## Signal APIs

Signals hold a state value that you can directly read and modify. You initialize them using the `signal()` factory function also known as Signal APIs.

- **Read a value:** Call the signal variable like a function (e.g., `count()`).
- **Directly set a value:** Use the `.set(newValue)` method to replace the state.
- **Update based on previous state:** Use the `.update(fn)` method, passing a function that computes the new value from the old value.

To understand create one component in your application:

**ng generate component components\signalcomponent**

Try following code in `signal-component.ts`

```
export class SignalComponent {  
  count = signal(0);  
  getCount() { return this.count(); }  
  ChangeMe() { this.count.set(10); }  
  UpdateMe() { this.count.update(value => value + 1); }  
}
```

A Signal is a reactive container that holds a value.

`count = signal(0);`

The important difference is that `count` is no longer a normal property. You **read** it using:

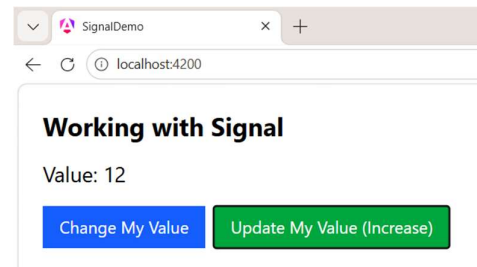
`count()`

You might change it using methods such as:

- `count.set(10);`
- `count.update(value => value + 1);`

`signal-component.html`

```
<div class="p-6">  
  <h2 class="text-2xl font-bold mb-4">Working with Signal</h2>  
  <h3 class="text-xl mb-4">Value: {{ count() }}</h3>  
  <button class="bg-blue-600 text-white px-4 py-2 mr-2"  
(click)="ChangeMe()"> Change My Value </button>  
  <button class="bg-green-600 text-white px-4 py-2 mr-2"  
(click)="UpdateMe()">  
    Update My Value (Increase)  
  </button>  
</div>
```





# Angular a Web Framework



**Web Framework: Crafted for Users, Powered by Google**

Rather than creating each Signal API using unrelated examples, better I recommend building one application for "All in One" kind of App.

So, let's walk through how to create and use signals.

Create default Angular Application:

**ng new signal-demo**

For styling, add tailwind CSS using Automated Setup with command:

**ng add tailwindcss**

This command performs the following actions:

- Installs tailwindcss and its peer dependencies.
- Configures the project to use Tailwind CSS.
- Adds the Tailwind CSS @import statement to your styles.

Test application:

**ng s -o**

**SIGNAL #1 – First Component — Counter**

**ng generate component components\counter**

Add in **counter.ts**

```
export class Counter {  
  count = signal(0);  
  increment() {  
    this.count.update(value => value + 1);  
  }  
  decrement() {  
    this.count.update(value => value - 1);  
  }  
  reset() {  
    this.count.set(0);  
  }  
}
```

Modify template counter.html

```
<p>Counter Component</p>  
<div class="p-6">  
  <h2 class="text-2xl font-bold mb-4">Counter</h2>  
  <p class="text-xl mb-4">Count: {{ count() }}</p>
```





# Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
<button class="bg-blue-600 text-white px-4 py-2 mr-2"
(click)="increment()">+</button>
<button class="bg-red-600 text-white px-4 py-2 mr-2"
(click)="decrement()">-</button>
<button class="bg-gray-600 text-white px-4 py-2"
(click)="reset()">Reset</button>
</div>
```

## SIGNAL #2 – Computed Signals (Derived State)

Oftentimes we have variables in our code that depend on other variables. For example, the total price for an item is the price for that item times the desired quantity of that item. If the user changes the quantity, we want to change the total price. For that, we use **computed signals**.

Computed signals are read-only signals that derive their value dynamically from other signals and they perform:

- **Automatic dependency tracking:** Angular automatically tracks which signals are read inside the computed function.
- **Lazy evaluation & Memoization:** The derivation function only executes when the computed signal is read, and its value is cached until the underlying source signals change.
  - **Memoization** is a programming optimization technique that stores the results of expensive function calls in a cache and reuses them when the same inputs occur again.

You can define them using the `computed()` function.

```
export class ComputedSignal {
  firstName = signal('Ravi');
  lastName = signal('Kumar');
  fullName = computed(() =>
    this.firstName() + ' ' + this.lastName()
  );
  constructor() {
    console.log(this.fullName()) //Ravi Kumar

    this.firstName.set('Alok');
    console.log(this.fullName()) //Alok Kumar
  }
}
```

Let's have a realistic example for computed signal, create a new component in existing application:

**ng generate component components/employee-statistics**

Add following to `employee-statistics.ts`



Presented by  
*Ranjan Bhatnagar*



# Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
import { Component, computed, signal } from '@angular/core';
export class EmployeeStatistics {
  employees = signal([
    { id: 1, name: 'John', department: 'IT' },
    { id: 2, name: 'Priya', department: 'HR' },
    { id: 3, name: 'Rahul', department: 'IT' },
    { id: 4, name: 'Anita', department: 'Finance' }
  ]);

  totalEmployees = computed(() =>
    this.employees().length
  );

  itEmployees = computed(() =>
    this.employees()
      .filter(e => e.department === 'IT')
      .length
  );

  hrEmployees = computed(() =>
    this.employees()
      .filter(e => e.department === 'HR')
      .length
  );
}
```

Modify template i.e. employee-statistics.html

```
<div class="p-6">
  <h2 class="text-2xl font-bold mb-4">Employee Statistics</h2>
  <div class="grid grid-cols-3 gap-4">
    <div class="p-4 bg-blue-100">
      <h3>Total Employees</h3>
      <p class="text-2xl">
        {{ totalEmployees() }}
      </p>
    </div>
    <div class="p-4 bg-green-100">
      <h3>IT Employees</h3>
      <p class="text-2xl">
        {{ itEmployees() }}
      </p>
    </div>
  </div>
</div>
```







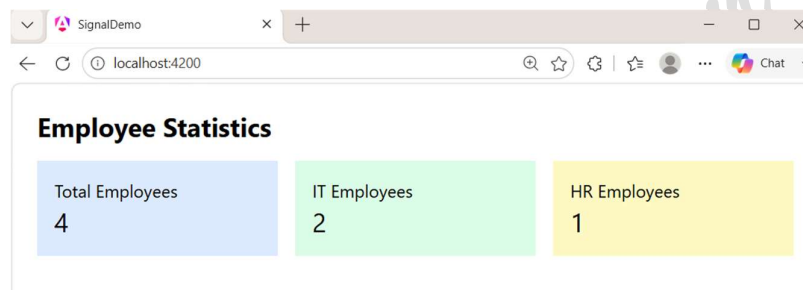
# Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
</p>
</div>
<div class="p-4 bg-yellow-100">
  <h3>HR Employees</h3>
  <p class="text-2xl">
    {{ hrEmployees() }}
  </p>
</div>
</div>
</div>
```

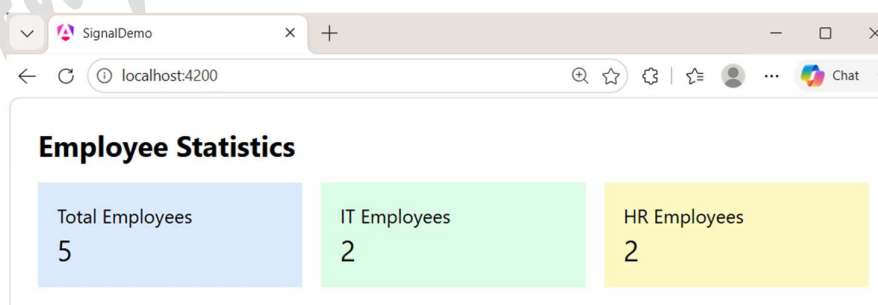
Observe an output:



Try adding one more Employee to static data in employee-statistics.ts

```
employees = signal([
  { id: 1, name: 'John', department: 'IT' },
  { id: 2, name: 'Priya', department: 'HR' },
  { id: 3, name: 'Rahul', department: 'IT' },
  { id: 4, name: 'Anita', department: 'Finance' },
  { id: 5, name: 'Sunil', department: 'HR' } //New Record
]);
```

Observe an output:



Imagine, an array → employees = signal<Employee[]>([]); We don't want to manually maintain:





# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**



- totalEmployees
- itEmployees
- hrEmployees
- financeEmployees etc.

Instead, derived values can be created as

- totalEmployees = computed(...);
- itEmployees = computed(...);
- hrEmployees = computed(...);

Angular understands their dependencies. totalEmployees, itEmployees, etc. are dependent on employees(), so, if employees changes, Angular knows which computed values depend on it.

Beware, computed() is Read-Only, and it is important, you shouldn't do:

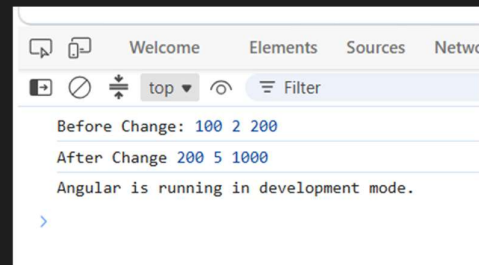
```
this.totalEmployees.set(20);
```

A computed signal is derived state. Instead:

Source State → computed() → Derived State

Another example:

```
export class PriceComputed {
  price = signal(100);
  quantity = signal(2);
  total = computed(() =>
    this.price() * this.quantity()
  );
  constructor() {
    console.log("Before Change:", this.price(), this.quantity(),
this.total());
    this.price.set(200);
    this.quantity.set(5);
    console.log("After Change", this.price(), this.quantity(),
this.total());
  }
}
```



## SIGNAL #3 – effect()

An **effect()** is an operation that runs **side-effect** logic whenever the signals it reads change their values. Effects always execute asynchronously during the change detection process and run at least once initially.





# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**

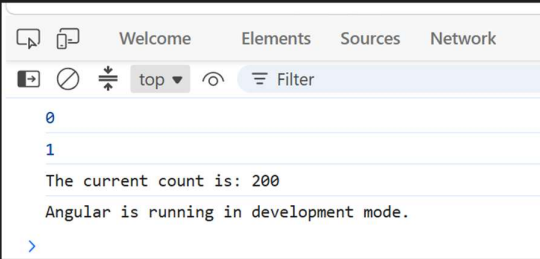


They are best utilized for:

- Logging data changes
- Syncing state with localStorage or external APIs
- Custom DOM manipulations outside of standard template rendering

Let's look at the code below and observe the console:

```
export class CountEffect {
  count = signal(0);
  constructor() {
    console.log(this.count());
    effect(() => {
      console.log(`The current count is: ${this.count()}`); // Fires
    });
    this.changeMe();
    console.log(this.count());
    this.count.set(200);
  }
  changeMe() {
    this.count.update(c => c += 1);
  }
}
```



Whenever count() changes, the effect runs.

But there is an important Angular detail here: effect() **does not necessarily execute synchronously at the exact line where you create it**. Effects are scheduled by Angular. The exact ordering can depend on Angular's execution/scheduling context.

Consider code below and observe console output:

```
export class CountEffect {
  count = signal(0);
  effectRuns = 0;
  constructor() {
    console.log('before effect:', this.count());
    effect(() => {
      this.effectRuns++;
      console.log(
        `EFFECT CALLED #${this.effectRuns}, count = ${this.count()}`
      );
    });
  }
}
```





# Angular a Web Framework

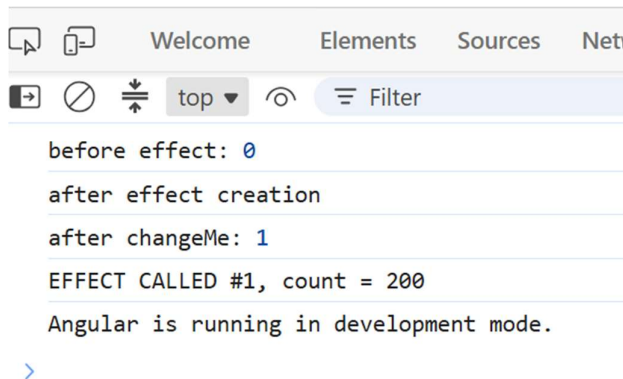
Web Framework: Crafted for Users, Powered by Google



```

console.log('after effect creation');
this.changeMe();
console.log('after changeMe:', this.count());
this.count.set(200);
}
changeMe() {
  this.count.update(c => c + 1);
}
}

```



In fact, console output is correct, and it demonstrates an important detail about Angular effect(). But the key point is: effect() is scheduled, not executed immediately when you call effect(...).

As per Angular's documentation, effects execute asynchronously during Angular's change-detection/synchronization process. By the time Angular actually executes the

effect, the current value is already 200.

## What happens in the code?

Our code effectively does this→

This is why your effectRuns counter says only:

EFFECT CALLED #1

Even when the effect should practically execute on each change. To prove it, try deliberately creating a delay between changes:

```

count = 0
create effect
↓
effect is scheduled
↓
changeMe()
↓
count = 1
↓
count.set(200)
↓
count = 200
↓
Angular runs the scheduled effect
↓
effect reads count()
↓
200

```

```

export class CountEffect {
  count = signal(0);
  constructor() {
    effect(() => {
      console.log(`EFFECT CALLED, count = ${this.count()}`);
    });
    setTimeout(() => {
      console.log('setting 1');
      this.count.set(1);
    }, 1000);
  }
}

```





# Angular a Web Framework

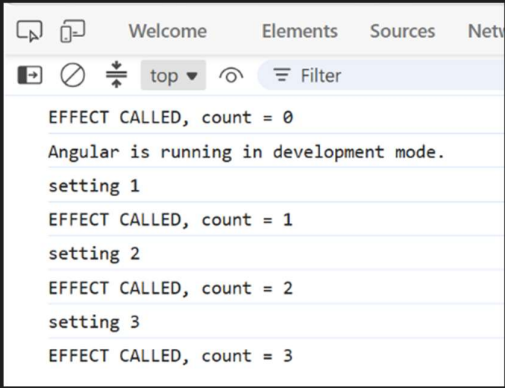
Web Framework: Crafted for Users, Powered by Google



```
    }, 1000);

    setTimeout(() => {
      console.log('setting 2');
      this.count.set(2);
    }, 2000);

    setTimeout(() => {
      console.log('setting 3');
      this.count.set(3);
    }, 3000);
  }
}
```



Now you can clearly see the effect running again for each change.

## One subtle but important distinction

As Angular says, `effect()` means:

"Run this side effect when the signals that this effect depends on change."

It does **not** mean:

"Execute this callback synchronously for every `.set()` or `.update()`."

Angular tracks the signals read during an effect, and schedules the effect when those dependencies change. Effects are specifically designed for side effects such as logging, syncing to `localStorage`, or interacting with non-signal APIs.

So, our original example is actually a very good demonstration of Angular's scheduling behavior. Our result is not evidence that `changeMe()` failed to trigger the effect. It triggered/scheduled the effect, but the effect hadn't run yet when we subsequently changed `count` to 200.

Now, create more useful example "Theme Switcher Example".

**ng generate component components/theme-switcher**

Now in `theme-switcher.ts`

```
export class ThemeSwitcher {
  darkMode = signal(false);
  constructor() {
    effect(() => {
      console.log(
        'Theme changed:',

```





# Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
this.darkMode() ? 'Dark' : 'Light'
);
});
}
toggleTheme() {
  this.darkMode.update(value => !value);
}
}
```

Modify template theme-switcher.html

```
<div class="p-6">
  <h2 class="text-2xl font-bold mb-4">Theme Switcher</h2>
  <p>
    Current Theme:
    {{ darkMode() ? 'Dark' : 'Light' }}
  </p>
  <button class="bg-indigo-600 text-white px-4 py-2 mt-4"
(click)="toggleTheme()">
    Toggle Theme
  </button>
</div>
```

Try clicking button and also observe console.

Important note about effect(): **Don't use effect() simply to calculate another value.**

- An effect() is meant for **side effects**, not for deriving state.
- computed() is better:
  - as computed() represents derived state.
  - Angular tracks dependencies automatically.
  - It avoids unnecessary writable state.
  - computed() is lazy and memoized. Angular can cache the computed value and recalculate it when its dependencies change.

effect() is intended for side effects or synchronizing with something outside Angular.

So, deduced,

"I need to CALCULATE something from signals" → computed()

"I need to DO something when signals change" → effect()

The key idea is **derived state belongs in computed(), side effects belong in effect()**.





# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**



Signals work naturally with Angular's modern control-flow syntax. Below given code is an excellent example to demonstrate how Signals integrate with modern Angular templates. Now let's combine: **Signal, Computed, @for and @if**. Add a component to application

ng generate component components/employee-search

In employee-search.ts

```
export class EmployeeSearch {
  searchText = signal('');
  employees = signal([
    { id: 1, name: 'John', department: 'IT' },
    { id: 2, name: 'Priya', department: 'HR' },
    { id: 3, name: 'Rahul', department: 'Finance' },
    { id: 4, name: 'Anita', department: 'IT' }
  ]);
  filteredEmployees = computed(() => {
    const search = this.searchText().toLowerCase();
    return this.employees().filter(employee => employee.name
      .toLowerCase()
      .includes(search)
    );
  });
}
```

Respective template employee-search.html

```
<div class="p-6">
  <h2 class="text-2xl font-bold mb-4">Employee Search</h2>
  <input
    type="text"
    placeholder="Search employee"
    class="border p-2 w-full mb-4"
    [value]="searchText()"
    (input)="searchText.set($any($event.target).value)"
  />
  @if (filteredEmployees().length > 0) {
    <div>
      @for (employee of filteredEmployees(); track employee.id) {
        <div class="border p-3 mb-2">
          <strong>
            {{ employee.name }}
          </strong>
        </div>
      }
    </div>
  }
```



# Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
<span class="ml-4">
  {{ employee.department }}
</span>
</div>
}
</div>
} @else {
  <p>No employees found.</p>
}
</div>
```

## Understanding Flow:

1. employees signal  
↓  
Contains all employees
2. searchText signal  
↓  
Contains what the user typed
3. computed() watches both signals  
↓  
Gets searchText  
↓  
Converts it to lowercase  
↓  
Gets employees  
↓  
Filters employees by name
4. filteredEmployees  
↓  
Contains only matching employees

searchText and employees are **signals that hold state**, while filteredEmployees is a **computed signal that automatically derives the filtered employee list whenever either dependency changes**.

If user types 'ra' then flow:

```
searchText = "ra"
↓
computed() runs
↓
"ra".toLowerCase()
↓
"ra"
↓
Check employee names
↓
Priya ✗
Rahul ✓
John ✗
Anita ✗
↓
filteredEmployees = [Rahul]
```

searchText changes  
↓  
computed() automatically recalculates  
↓  
filteredEmployees changes

You **don't** manually call the **computed** function to update it, and you **don't** need effect().

The simple formula is: **Input signals → computed() → derived result**

## Browser Output:

SignalDemo

localhost:4200

### Employee Search

|       |         |
|-------|---------|
| Rahul | Finance |
|-------|---------|







# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**



## Signals and Component Communication

Modern Angular builds upon signals to streamline component data flows, moving away from classic decorator patterns. Signals become even more interesting when we use them between components. Modern Angular provides signal-based APIs such as:

- **input() / InputSignal:** Replaces the legacy @Input() decorator. It receives read-only properties from parent components as signals.
- **output() / OutputSignal:** Replaces @Output() and EventEmitter with a leaner approach to emitting events from child components.
- **model() / ModelSignal:** Enables two-way data binding. A model() can be written to by both the parent and child components.

These provide a modern approach to component communication.

## Signal Input and Signal Output Example

Generate two components:

ng generate component learning/ParentComponent

ng generate component learning/ChildComponent

In parent-component.ts

```
export class ParentComponent {  
  parentMessage = 'Hello from Parent!';  
  childMessage = '';  
}
```

In parent-component.html

```
<div class="min-h-screen bg-gray-100 p-8">  
  <div class="mx-auto max-w-2xl">  
    <div class="rounded-xl bg-white p-6 shadow-lg">  
      <h2 class="mb-2 text-2xl font-bold text-gray-800">Parent  
Component</h2>  
  
      <p class="mb-6 text-gray-500">Parent → Child → Parent  
communication</p>  
  
      <!-- Message received from Child -->  
      <div class="mb-6 rounded-lg bg-green-50 p-4">  
        <p class="text-sm font-medium text-green-700">Message from  
Child</p>
```





# Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
<p class="mt-1 text-lg text-green-900">
  {{ childMessage || 'No message yet' }}
</p>
</div>

<!-- Child Component -->
<app-child-component [message]="parentMessage"
(messageChange)="childMessage = $event">
  </app-child-component>
</div>
</div>
</div>
```

In child-component.ts

```
export class ChildComponent {
  // Signal Input
  message = input<string>();

  // Output for sending data back to parent
  messageChange = output<string>();

  sendMessage() {
    this.messageChange.emit('Hello from Child!');
  }
}
```

In child-component.html

```
<div class="rounded-lg border border-gray-200 bg-gray-50 p-5">
  <h3 class="mb-4 text-xl font-semibold text-gray-800">Child
Component</h3>

  <!-- Signal Input -->
  <div class="mb-5 rounded-lg bg-blue-50 p-4">
    <p class="text-sm font-medium text-blue-600">Message from
Parent</p>

    <p class="mt-1 text-lg font-medium text-blue-900">
      {{ message() }}
    </p>
```





# Angular a Web Framework

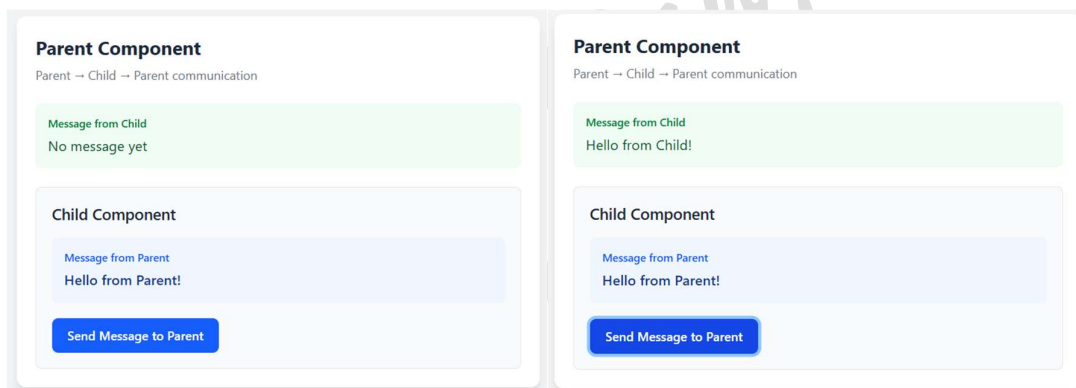
Web Framework: Crafted for Users, Powered by Google



```
</div>

<!-- Send message to Parent -->
<button
  type="button"
  (click)="sendMessage()"
  class="rounded-lg bg-blue-600 px-5 py-2.5 font-medium text-white
transition hover:bg-blue-700 focus:outline-none focus:ring-4
focus:ring-blue-300"
>
  Send Message to Parent
</button>
</div>
```

Output: before and after clicking button



In Angular, the **Parent component** passes data to the **Child component** using a Signal Input such as `input()`, allowing the child to receive and display the parent's data. The **Child component** can send data back to the Parent using `output()` and `emit()`.

This creates a simple **Parent → Child → Parent** communication flow, where the parent controls the data and the child can notify the parent about changes or user actions.

**Brief comparison: Signal input()Traditional @Input()**

| Feature        | Signal input()                     | Traditional @Input()                   |
|----------------|------------------------------------|----------------------------------------|
| Syntax         | <code>input&lt;string&gt;()</code> | <code>@Input() message!: string</code> |
| Type safety    | Better inference                   | Good, but more manual                  |
| Reactive       | Signal-based                       | Regular property                       |
| Reading value  | <code>message()</code>             | <code>message</code>                   |
| Modern Angular | ✅ Recommended                      | ⚠️ Legacy/older style                  |
| Migration      | New Angular APIs                   | Still supported                        |



Presented by  
*Ranjan Bhatnagar*



# Angular a Web Framework



**Web Framework: Crafted for Users, Powered by Google**

Brief comparison: Signal Output() Traditional @Output()

| Feature        | Signal output()    | Traditional @Output()                          |
|----------------|--------------------|------------------------------------------------|
| Syntax         | output<string>()   | @Output() message = new EventEmitter<string>() |
| Emits value    | .emit(value)       | .emit(value)                                   |
| Reactive API   | Modern Angular API | EventEmitter-based                             |
| Simplicity     | Less boilerplate   | More boilerplate                               |
| Modern Angular | ✓ Recommended      | ⚠ Still supported                              |

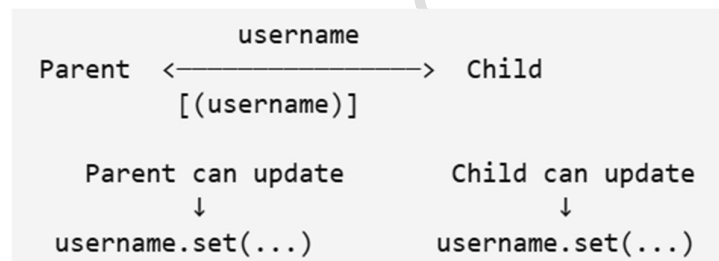
## Finally

For **new Angular applications**, input() and output() are generally the better choice because they provide a cleaner, more modern API with less boilerplate and better integration with Angular's signal-based reactivity. However, @Input() and @Output() are **still supported**, so there is no need to rewrite existing applications just for the sake of migration.

## model() / ModelSignal example

Here given is a real-world example where **parent controlling a child's username input** with model(), both the parent and child can update the same value using Angular's two-way binding syntax [(username)].

The model() creates a **ModelSignal**, which can be **read and written** by the child. This creates two-way binding:



In the same application I am creating two more components:

ng generate component learning\ParentModelComp

ng generate component learning\ChildModelComp

In parent-model-comp.ts

```

export class ParentModelComp {
  username = signal('John');

  setFromParent() {
    this.username.set('Updated by Parent');
  }
}

```



# Angular a Web Framework

**Web Framework: Crafted for Users, Powered by Google**



In parent-model-comp.html

```
<div class="max-w-lg mx-auto mt-10 rounded-xl bg-white p-6 shadow">
  <h2 class="text-2xl font-bold text-gray-800">Parent Component</h2>

  <p class="mt-2 text-gray-600">
    Username from Child:
    <span class="font-semibold text-blue-600">
      {{ username() }}
    </span>
  </p>

  <!-- Two-way binding -->
  <app-child-model-comp [(username)]="username" />

  <button
    class="mt-4 rounded-lg bg-green-600 px-4 py-2 text-white
    hover:bg-green-700"
    (click)="setFromParent()"
  >
    Update From Parent
  </button>
</div>
```

In child-model-comp.ts

```
export class ChildModelComp {
  // ModelSignal
  username = model<string>('');
}
```

In child-model-comp.html

```
<div class="mt-6 rounded-lg border bg-gray-50 p-5">

  <h3 class="text-xl font-semibold text-gray-800">
    Child Component
  </h3>

  <input
    class="mt-4 w-full rounded-lg border border-gray-300
    px-4 py-2 focus:border-blue-500 focus:ring-2
```





# Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
        focus:ring-blue-200"
        [value]="username()"
        (input)="username.set($any($event.target).value)"
        placeholder="Enter username"
    />

    <p class="mt-3 text-gray-600">
        Current value:
        <span class="font-semibold">
            {{ username() }}
        </span>
    </p>
</div>
```

Output:

**Parent Component**  
Username from Child: John

**Child Component**  
John  
Current value: John

Update From Parent

Changing Text

**Parent Component**  
Username from Child: India

**Child Component**  
India  
Current value: India

Update From Parent

After Clicking Button

**Parent Component**  
Username from Child: Updated by Parent

**Child Component**  
Updated by Parent  
Current value: Updated by Parent

Update From Parent

model() in Angular is used to create a **ModelSignal** that supports **two-way data binding** between a parent and child component. In the example, `username = model<string>()` allows the child to both **read** the parent's username and **update** it. Using `[(username)]="username"` in the parent keeps both components synchronized automatically. This is useful when a child component needs to modify a value that is owned by the parent.

